CLMSynth v0.7.5rc1

Arootin Gharibian

October 29, 2026

Abstract

CLMSynth generates a label $L(x)$ and attaches it to an existing set of cluster IDs $c(x)$ (ground truth or algorithm output), with the relationship between L and c specified by a configuration file and then measured afterwards. A synthetic dataset generated with this program can be used to benchmark and evaluate cluster and label matching under user-definable parameters: clusters with random class assignments and balanced or imbalanced clusters with varying degrees of cluster-label matching. Unlike existing clustering benchmarks that implicitly equate clusters with classes, the generator treats cluster formation and class assignment as independent processes with configurable matching outcomes. Each dataset instance includes ground-truth cluster membership, class labels, and metrics to create reproducible comparative benchmarking of cluster-label matching. A flexible target-metric solver can iterate over possible configurations, respecting user-defined criteria for label assignment, and solve for a CLM function within tolerance of a specific target.

License

CLMSynth Documentation is licensed under Creative Commons Attribution-ShareAlike 4.0 International license. The source code for the program is published under the MIT License at GitHub.

1 Introduction

Existing synthetic datasets for clustering performance measurement assume equivalence between latent cluster structure and assigned class labels. The CLM dataset synthesizer addresses the natural occurrence of classes within latent clusters by treating cluster generation and class assignment as independent, configurable, and target-matched for a specific cluster-label matching. This design supports multiple benchmarks, including non-clusterable data with strong class separability, well-defined clusters with randomly assigned classes, and both balanced and imbalanced clusters. For each configuration, the output is a dataset of labels attached to the cluster membership, along with summary diagnostics characterizing cluster-label matching (CLM) measures. By generating and systematically comparing reproducible CLM scenarios, this resource complements existing clustering benchmarks and supports the evaluation of methods that study the relationships between the clustering structure and class labels.

1.1 Scientific Objectives

Cluster-label matching (CLM) is measured as the feature subset's effectiveness in predicting ground-truth class labels, the degree to which a feature subset supports known classes or labels regardless of clustering performance. The performance and quality outcomes associated with a selected subset of features, evaluated in terms of its ability to detect structures (clusters) and its alignment with class labels, are independent. Different algorithms and measures can optimize the selection of a given subset of features and assess how it performs in clustering, classification, and cluster-label matching. The objective of this program is to create datasets that give a user granular control over labels and match specific cluster-label matching targets, independent of the clustering, so that an algorithm or measure can reproduce specific CLM targets.

1.2 Implementation

The implementation in Python, together with the pipeline scripts, configuration templates, and working examples, is maintained in the project repository. The source code is the authoritative description of the implementation; this manual is intended for software users and includes the definition of the configuration schema and the troubleshooting reference.

2 Implementation Details

2.1 Label generator

1. **Validate**, structural checks fail fast, before any label sizing: the label count `num_classes` must lie in $[1, 64]$; `matching_mode` must be one of the four modes, checked before every other key because the mode decides which keys are read; any requested spatial placement requires per-point coordinates; the data's own cluster count K must lie in $[1, 64]$ (a coarse backstop rather than a reliability boundary); `perfect` requires $M = K$. Outside `perfect`, `balance` must be exactly `balanced` or `unbalanced`, and `skew_params` must be in range for the rule that reads them. The mode's own keys are checked once the counts are resolved: `single` requires `single_match`, $M \geq 2$, $K \geq 2$; `target_metric` is only compatible with `single`/`custom`, and `scope: pair` additionally requires `type: mcc` and `matching_mode: single`.

2. **Resolve proportions** p are uniform if `balanced` (the default when `balance` is absent), and skipped under `perfect`, whose counts are the cluster sizes; otherwise, `proportions` take precedence, with `skew_rule` as a fallback.

3. **Matching mode** $\rightarrow$ alignment rules:

   - `random`, a random permutation of the fixed multiset holding each label ℓ_i exactly m_i times, rather than an independent draw from p per point: the realized counts equal m_i, and the permutation consults neither $c(x)$ nor position.
   - `perfect`, bijection π : cluster $\rightarrow$ label in sorted order; label counts are assigned to the paired cluster sizes; $L(x) = \pi(c(x))$.
   - `single`, one target pair (k^*, ℓ^*) aligned at full recall; all other points are assigned by spillover.
   - `custom`, an `assignment_matrix` of rules, each a `recall_target` fraction of one label's point budget into a chosen set of clusters (enabling surjective, partial, and overlapping).

4. **Allocation & feasibility**, each rule is divided into its clusters (`split_rule`); exceeding the cluster capacity raises an infeasibility error stating the maximum feasible recall (under `single`, the most points the label can have) and the setting the label's size comes from. Counts are computed from the values as written (0.7 is 7/10) and rounded to whole

points by largest remainder, a tie going to the earlier label, rule or entry; each rule's count may be its claim rounded down or up, so a rule or cluster is infeasible only when no such rounding of the rules fits. The label counts are rounded before the rules, on their own. The remaining cluster capacity goes to `competing_noise` first and is then filled by `spillover_rule`. The claims follow one order: the label counts, the matching (at the solved recall level under `target_metric`), `competing_noise`, spillover. `proportional_to_marginal` preserves the target proportions: a label's `competing_noise` entries together place at most what the matching leaves of its count. The summary txt lists, for each label, the points every `assignment_matrix` row and `competing_noise` entry asks for and the points it took.

5. **Target metric**: the target metric and the solver vary only the recall level; every setting above is held fixed across probes, so the noise structure changes the solved recall rather than being applied after it.

   With `scope: global` (the default), one recall level is solved for the whole partition by a coarse grid scan; with `scope: pair`, the single-pair MCC of the `single_match` pair is inverted exactly and no search runs. A target above the configuration's reachable ceiling or below its floor terminates at the closest achievable value, which is delivered unless `strict: true` is set; then a value outside the tolerance is refused (`[CLM-154]`) and the label is not written. Under `scope: global` each run also reports the highest value the search found and, when nothing can move the label counts, their size ceiling: the highest value any labeling with those cluster and label counts can score (for ARI, a bound that is not always reached). A `uniform` or `concentrated` spillover can move the counts, and then no size ceiling is reported. Both values are logged, returned through the `report` argument of `generate_clm_labels` and written to the summary txt; neither stops the search. Because R_K is a maximum over matching, and the mean of R_K over all relabeling is exactly 0, the matched value is never negative and its expectation under independent labeling is strictly positive.

   Currently, three CLM measures are available: `clustering_mcc_pair` is the 2×2 Matthews ϕ of a cluster against a label, which does not need matching. `clustering_mcc` is the permutation-invariant `scikit-learn` implementation of Gorodkin's R_K, which is a one-to-one matching of the cluster–label contingency table that maximizes R_K. A solution that converged internally can still land outside the tolerance, most visible at small N, where the placement of spillovers is a large part of the result.

`clustering_ari` is solved for the adjusted Rand index (ARI): The ARI measures the agreement over *pairs* of points rather than through any correspondence between the label and cluster identifiers, and is chance-corrected so that independent partitions score 0. Because it is a function of the cell of the contingency table and the marginal pair counts alone, it does not need an alignment step: permuting the values of the generated label leaves it unchanged, it is symmetric in its two arguments, and it is defined for any M and K.

6. **Spatial placement** Configures optionally, within each cluster, *which* points receive each label is chosen by weighted sampling without replacement, with weights derived from normalized distance to the cluster centroid, with`profile`: `linear`/`step`/`exponential`; and `favors`: `steepness` for exponential or `core`/`boundary` for other profiles. The target CLM outcome is fixed by stages 2–5, so spatial structure can be varied at a constant metric value.

Note on multi-class multi-label measurement: Releases of up to 0.6.9 were matched on the grid-search to maximize precision, and thus reported the R_K of accuracy-optimal matching. The difference is only near the lower end of the target metric, visible for unbalanced datasets. From release 0.7.5 CLM-306 reports a target the search missed, and the closest label is still delivered (with `target_metric.strict: true`, CLM-154 refuses it); at small N the reachable values form a coarse ladder and a tight tolerance can fall between intervals. Since 0.6.4 the search and the final generation share one random stream, so the scored labeling is the written one; only an explicit `target_metric.probe_seed` separates them, which is what CLM-309 reports.
Note on Spatial placement: `centroid_dependence` or a `competing_noise` entry that favors the core or the boundary) requires per-point feature vectors. The complete catalog of coded errors and warnings is the reference for troubleshooting.

2.2 Configuration file definition

The `clm_label_engine` is the core module for label synthesis. The configuration parameters for the core module are defined as follows:

```
clm_label:
  num_classes: M                    # 1 <= M <= 64; the data's
    own K is capped
  balance: balanced | unbalanced    # balanced: uniform 1/M,
   proportions ignored.
  proportions: [...]                # unbalanced: used
   directly when given.
  skew_rule: geometric | dominant_minority | dirichlet
                                    # unbalanced fallback when
    no proportions
  skew_params:
    ratio: 0.5                      # geometric: p_i ~ ratio^i
    dominant_index: 0               # dominant_minority
    dominant_share: 0.5             # dominant_minority
    alpha: 1.0                      # dirichlet: Dirichlet
  matching_mode: perfect | single | random | custom
  single_match: {cluster: k*, label: l*}
  assignment_matrix:
    - {label: l, clusters: [k1, ...], recall_target: r}
                                    # each cluster named once
   per label (CLM-142)
  split_rule: proportional_to_size | equal   # how a rule's
   mass splits over its clusters
  spillover_rule: proportional_to_marginal | uniform |
   concentrated
  concentrated_labels: [l, ...]     # spillover_rule:
   concentrated only.
  target_metric:                    # optional (single/custom,
    M and K >= 2)
    type: mcc | ari
    value: 0.7
    scope: global | pair            # global (default): one
   recall solved
                                    # pair: exact, for the
   single_match
                                    #   pair; mcc only.
    tolerance: 0.01                 # both scopes: how far the
    DELIVERED
                                    #   labeling may sit from
   the target.
    max_iter: 40                    # global only, bisection
   budget; the pair
                                    #   scope inverts exactly
   and never iterates
```

```
  probe_seed: 42                  # optional; defaults to
 the run seed
  strict: true | false            # optional (false): true
 refuses a label
                                  #   outside tolerance (CLM
 -154); tolerance
                                  #   then defaults to 0,
 the exact value
competing_noise:                  # optional (single/custom)
 : structured noise.
  - cluster: k
    label: l_comp
    share: 1.0                    # fraction of that cluster
 's leftover points,
                                  #   at most what l_comp
 has left (CLM-305)
    favors: boundary | core | random
centroid_dependence:
  enabled: true | false
  profile: linear | exponential | step
  favors: core | boundary
  steepness: 3.0                  # exponential only
```

3 Usage

The input variables required for the generator (`X` and `c`; cluster centroids are derived internally from `X`) are expected to be supplied by external clustered datasets. CLMSynth is designed to work with online and offline datasets, such as the Gagolewski's clustering benchmark suite (`clustbench`) or synthetic datasets generated via MDCGen (`mdcgen`). In addition, synthetic data generated through the offline generator (`fabricated_data`) and direct import through `byoc` (*bring-your-own-clusters*) are possible.
The package installs three entry points: `clmsynth` runs the pipeline with a YAML or JSON configuration; `clmsynth-config` renders a configuration from an upstream payload, and `clmsynth-wizard` generates one interactively. A simple demonstrative run is as follows:

```
# Install package with requirements, then:
from clmsynth.clm_label_engine import generate_clm_labels
# An arbitrary clustering data:
rng = np.random.default_rng(0)
sizes = [200, 150, 150]
c = np.repeat([0, 1, 2], sizes)
X = np.vstack([rng.normal(mu, 1.0, size=(n, 2))
               for mu, n in zip([(0, 0), (6, 0), (0, 6)],
   sizes)])
# Basic configuration:
config = {
    "num_classes": 3,
    "balance": "unbalanced",
    "proportions": [0.5, 0.3, 0.2],
    "matching_mode": "custom",
    "assignment_matrix": [
        {"clusters": [1, 2], "label": 0, "recall_target":
   0.9},
    ],
    "split_rule": "proportional_to_size",
    "spillover_rule": "proportional_to_marginal",
    "centroid_dependence": {
        "enabled": True,
        "profile": "exponential",
        "favors": "core",
        "steepness": 4.0,
    },
}
# Generate labels:
L = generate_clm_labels(cluster_labels=c, coords=X, cfg=
   config, seed=42)
print(L.value_counts().sort_index().to_dict())
# -> {0: 250, 1: 150, 2: 100}
```

4 Example Outputs

The global R_K/ARI between an M-label and a K-cluster partition has no closed form, which is why `scope: global` is solved numerically. The compact closed-form expression $TP^\star$ inverts the *single* confusion table MCC (what `scope: pair` evaluates), which is why that mode reaches its target without a search. Maximizing over matchings lifts R_K's null expectation above 0, while ARI needs no alignment and stays centered on it (config A2). That is why the CLM target for single-match configurations A3 and D1 is feasible in table 1. Configurations A4 and B1–B3 share one set of rules (labels sized $[1024, 614, 410]$, one rule per label); B1 and B2 request possible values within their configuration (solver within the tolerance 0.01), while B3, a 3-label partition of a 2-cluster, requests MCC = 0.99, which is not possible. The solver returns the closest feasible value, 0.718, the maximum attainable for this configuration. C1–C3 differ only in spatial placement.

Cfg	Configuration	Expected	Actual
A1	perfect, M=K=2	bijection; MCC ≈ 1	MCC = 1.000, ARI = 1.000
A2	random, M=4	no cluster info; ARI ≈ 0, R_K at its null level	MCC = 0.018, ARI = −0.000
A3	single (cluster 1, M=4)	feasible here (512 < 1024); partial alignment	MCC = 0.279, ARI = 0.166
A4	custom, explicit recalls (0.8/0.5/0.3)	feasible; counts $[1024, 614, 410]$	MCC = 0.527, ARI = 0.393
B1	custom + solve MCC= 0.6	reachable here; solver hits it within tol.	solved MCC = 0.598 (target 0.6)
B2	custom + solve ARI= 0.4	reachable here; solver hits it within tol.	solved ARI = 0.391 (target 0.4)
B3	custom + solve MCC= 0.99, max_iter 15	unreachable; non-convergence + delivered-value check	solved MCC = 0.718
C1	single largest + centroid linear/core	MCC = A3 (placement in 4-D)	MCC = 0.279, ARI = 0.166
C2	single largest + centroid exp/boundary	same contingency as C1	MCC = 0.279, ARI = 0.166
C3	single largest + centroid step/-core	same contingency as C1	MCC = 0.279, ARI = 0.166
D1	single (cluster 1, M=2)	fills cluster exactly; MCC ≈ 1	MCC = 1.000, ARI = 1.000

Table 1: Cluster–label matching outcomes on `g2mg_4_20` ($N = 2048$, 4-D, $K = 2$ clusters of 1024 points, reference labeling `labels0`).

Revision History

Changes from after the published article, behavior, diagnostics and label allocation can diverge from the article, specifically, limitations are addressed. Up to publishing the package in conda-forge.

Revision	Date	Author(s)	Description
0.7.3	2026-10-11	AG	Reporting the target metric minimum and maximum, diagnostics and robustness.
0.7.4	2026-10-16	AG	Python 3.15 and batch runs initial support, seaborn removed, PyYAML optional with JSON configs.
0.7.5rc1	2026-10-29	AG	Better testing, deterministic exits and configs.
0.7.5	2026-10-29	AG	Cumulative release in PyPI with pipeline support, parallel runs, structural ceiling and rounding limitations addressed.

Revision History

From first release to the release matching the published article with label allocation and behavior, limitations and diagnostics matching the manuscript.
.

Revision	Date	Author(s)	Description
0.1	2026-07-07	AG	Initial release: CLM label engine, `[CLM-###]` diagnostics, configuration wizard, MCC/ARI evaluation.
0.2	2026-07-15	AG	Internal, unpublished.
0.3	2026-07-31	AG	`clustering_mcc_pair` added to the API; `fabricated_data` cluster ids changed from strings to integers.
0.4	2026-07-31	AG	The renderer can now emit `target_metric`, `competing_noise`, `concentrated_labels`, `skew_params` and `steepness`; `mdcgen` source repaired.
0.5	2026-07-31	AG	Research release: `CITATION.cff`, `[CLM-1xx]` errors abort the run.
0.6.2	2026-08-02	AG	First public release. Label-space and pair-scope guards: `[CLM-121]`, `[CLM-128]`, `[CLM-129]`, `[CLM-130]`, `[CLM-153]` and `[CLM-310]` reject configurations that previously produced silently incorrect labels; documentation revised throughout
0.6.3	2026-08-08	AG	Correctness release: `[CLM-131]` validates `skew_params`; `scope: pair` now honors `tolerance`; run folders are created atomically; `[CLM-104]`/`[CLM-105]` report per dataset instead of aborting a batch.
0.6.4	2026-08-09	AG	Tooling and import hygiene. Shared target-metric search and label generation stream. Validated `byoc` imports (uncoded rejections section
0.6.5	2026-08-10	AG	The test suite becomes part of the repository.
0.6.6	2026-08-11	AG	A patch release

0.6.7	2026-08-12	AG	CLI-Wizard update and improvement
0.6.8	2026-08-20	AG	Patch without release, preparation for CLMSynth-GUI
0.6.9	2026-08-23	AG	Patch release, staging for release 0.7.0
0.7.0b1	2026-08-30	AG	Article-review release: maximize the multiclass `R_K` rather than accuracy; Brent's method for the recall solver ; `byoc_suite.tag_columns` carries non-feature columns through an import; support for Python 3.11 dropped.
0.7.0rc1	2026-08-30	AG	Release candidate, rehearsed on TestPyPI. (TestPyPI)
0.7.0rc2	2026-09-01	AG	Documentation corrections: `[CLM-309]'` and `scope: pair` description. (TestPyPI, GitHub)
0.7.0	2026-09-01	AG	Release on PyPI. SoftwareX Article Version. (PyPI, GitHub)
0.7.1	2026-09-17	AG	Patch release: negative `proportions`, `recall_target`, `tolerance` and `steepness` are rejected; `[CLM-205]`/`[CLM-207]` report a missing `label`.
0.7.2	2026-10-04	AG	Patch release: `[CLM-132]` refuses other than `balanced`/`unbalanced`; `matching_mode`; `num_classes` must be an integer (`[CLM-126]`); `[CLM-133]` rejects if dataset's cluster ids or `coords` are unusable; `[CLM-134]` refuses a value of the wrong type. This release is the final release that matches the SoftwareX version, changes from this release can diverge from the article.

Licenses for Third-Party Libraries

Python is distributed under the Python Software Foundation License V.2. This project relies on several open-source third-party Python libraries, including `NumPy`, `Pandas`, `SciPy`, `scikit-learn`, and `Matplotlib`. Optionally `PyYAML` and `mdcgenpy`. These packages are distributed under their respective licenses (BSD 3-Clause, MIT, and Apache 2.0). Refer to the official documentation of each package or the project repository for full third-party license details.

5 References

1. Gagolewski, M. (2020). *A framework for benchmarking clustering algorithms*; Author's website: clustering-benchmarks

2. Iglesias, F., Zseby, T., Ferreira, D. *et al.* (2019). *MDCGen: A Multidimensional Dataset Generator for Clustering.* Journal of Classification. doi: 10.1007/s00357-019-9312-3

3. de Vries, S., Thierens, D. Generating the ground truth: Synthetic data for soft label and label noise research. Int J Data Sci Anal 20, 5603–5615 (2025). doi: 10.1007/s41060-025-00786-z

4. Jeon, H., Aupetit, M., Shin, D., Cho, A., Park, S., and Seo, J. (2025). *Measuring the Validity of Clustering Validation Datasets.* IEEE Transactions on Pattern Analysis and Machine Intelligence, 47(6), 5045-5058. doi: 10.1109/TPAMI.2025.3548011

5. Gharibian, A., Kudělka, M. (2026) A synthetic data generator for cluster and label matching evaluation. SoftwareX 36, 103077. doi:10.1016/j.softx.2026.103077.

Configuration Troubleshooting Reference

CLMSynth v0.7.5rc1

October 29, 2026

This reference documents every configuration condition that is either an incompatibility or an ignored setting, as well as the diagnostic message of the CLMSynth label engine. It is intended to help users *plan* a valid configuration and interpret what the engine does with a specific configuration.

How the code works. The identifiers below are a documentation index for this manual; the core label synthesizer engine raises standard Python exceptions with messages banded by type:

`1xx` **ValueError** invalid value or
Incompatible option (`101`–`149`, the two kinds interleave) or
InfeasibleAllocationError, Infeasible outcome (`150`–`169`).

`2xx` **MissingConfigKey** (a **KeyError**) a required configuration key is missing (message carries the code); it has no default, so the run stops.

`3xx` **Warning** non-fatal; logged via `logging.warning`, execution continues.

All errors have example configurations documented in the project repository; issues must be addressed after checking these invalid configurations and reported only if previously not covered.

1 Configuration option compatibility at a glance

Option	perfect	single	random	custom
`proportions` / `balance` / `skew_rule` / `skew_params`	○	✓	✓	✓
`single_match`	–	✓ req	–	–
`assignment_matrix`	–	–	–	✓ req
`split_rule`	○	○	–	✓
`spillover_rule`	○	✓	–	✓
`competing_noise`	○	✓	×	✓
`target_metric`	×	✓	×	✓
`target_metric.scope: pair`	–	✓	–	×
`centroid_dependence`	○	✓	○	✓

Table 1: Option & matching_mode compatibility: ✓ used / required (**ValueError** or **KeyError** if required but not provided), – not read / not applicable, ○ accepted but ignored or no visible effect (Table 8 lists which log a **Warning**; under `perfect` the label-sizing keys of the first row are not validated either; enabled placement still needs coordinates), × not allowed (raises a **ValueError** of type *Incompatible*).

Not every configuration rejection is diagnostic `CLM`. There are conditions that check whether the *data* that were imported by the configuration is a usable clustering at all. They do not carry a code because the solution is outside the scope of the program.

2 Requirements & Restrictions

2.1 Requirements (planning)

The user shall provide, through a configuration file, a required set of settings (Table 2). It is necessary to plan the required configuration before executing the label synthesizer; an independent CLI wizard module provides step-by-step guidance for generation of the configuration file.

Setting	Requires
`num_classes` (M)	Integer in $[1, 64]$ (≥ 2 under `single`); the dataset's own cluster count K is capped the same way (126/127).
`balance`	Exactly `balanced` or `unbalanced`, matched case-sensitively (132). Omit the key for the default, `balanced`; a valueless `balance:` is not the default and is refused too. Not read under `perfect`.
`unbalanced`	Either explicit `proportions` *or* a `skew_rule` (203). Explicit proportions need exactly `num_classes` entries (121) summing to 1 (106). Rules have defaults, so `skew_params` may be omitted.
`skew_params`	`skew_params` for its type: `geometric`→`ratio`; `dominant_minority`→`dominant_index`,`dominant_share`; `dirichlet`→`alpha`.
`matching_mode`	One of `perfect`, `single`, `random`, `custom`.
`perfect`	`num_classes` == number of clusters ($M = K$); a bijection needs one label per cluster. Nothing else: `balance`, `proportions`, `skew_rule` and `skew_params` are neither read nor validated (302).
`single`	$M \geq 2$, $K \geq 2$, and `single_match:{cluster,label}`; the cluster must exist and the label must be in $0..M-1$.
`custom`	`assignment_matrix` with ≥ 1 row; each row's `label`$\in 0..M-1$ and `clusters` exist; a cluster is named once per label across the matrix (142); needs `recall_target` *unless* `target_metric` is set.
`target_metric:`	`matching_mode` $\in$ {`single`, `custom`}, with $M \geq 2$ and $K \geq 2$ under `custom` (111); `type`$\in${`mcc`,`ari`}; `value`$\in [-1, 1]$.
`target_metric.scope`	`"global"` (default) or `"pair"`. `pair` requires `type: mcc` and `matching_mode: single`, solved in closed form within tolerance.
`competing_noise`	`matching_mode` $\in$ {`single`, `custom`}; each entry: `cluster` exists, `label`$\in 0..M-1$, `share`$\in [0, 1]$, `favors`$\in${`core`,`boundary`,`random`}; one entry per label and cluster (142), with a label that is not a target of that cluster (144). Needs *unclaimed* (leftover) points to have any effect; placed after the cluster's target labels.
`concentrated_labels`	Under `spillover_rule: concentrated` only: a *list* of integer label ids, each $\in 0..M-1$ (128). A bare number is not a shorthand for a one-element list. Default, takes the single largest label.
`spatial placement`	Per-point feature vectors (an (N, d) `coords` array) whenever `centroid_dependence` is enabled or a `competing_noise` entry favors `core`/`boundary`; labels-only configs must omit them.
`centroid_dependence`	`enabled` a boolean, `true` or `false` unquoted (141); `profile`$\in${`linear`,`exponential`,`step`}; `favors`$\in${`core`,`boundary`}; `steepness` only for `exponential`.

Table 2: List of configuration a user is required to plan and provide to the software

2.2 Restrictions

Incompatibilities are configurations in which every individual setting is valid but the combination is not: the requested features are mutually exclusive so that no assignment can satisfy both at once. Examples are explained below, the complete code list is in Table 5. Restricted configurations are covered by the `config_wizard` CLI utility; thus, to generate valid configurations, the wizard is the recommended tool.

Feature	Not allowed when	Code	Reason & fix
`target_metric`	`matching_mode = perfect`	111	Perfect is a fixed bijection (MCC= 1); there is no free `recall_target` to search. Use `single`/`custom`.
`target_metric`	`matching_mode = custom` with `num_classes` $= 1$ or one cluster	111	There is a single labeling, so every metric is fixed and nothing can be searched. Use $K, M \geq 2$.
`target_metric`	`matching_mode = random`	114	Random ignores clusters entirely; there is no agreement level to dial. Use `single`/`custom`.
`competing_noise`	`matching_mode = random`	115	There is no aligned label for a competing label to compete with. Use `single`/`custom`.
`target_metric.scope: pair`	`type = ari`	123	The single-pair MCC inverts in closed form; ARI has no inverse. Use `type: mcc`, or `scope: global`.
`target_metric.scope: pair`	`matching_mode` $\neq$ `single`	124[1]	The target pair is the `single_match`. Use `scope: global`.
`spatial placement`	`features` missing, empty or a single feature	125	Single-feature data is out of scope. Pass an (N, d) feature array with $d \geq 2$, or disable `centroid_dependence`.
`centroid_dependence.favors`	not `core`/`boundary`	129	Matching is case-sensitive and exact. Any other value, including a capitalization slip (`Core`) or a typo, is only read when `enabled: true`.
`target_metric.scope: pair`	`spillover_rule` is `uniform`; or `concentrated` targeting the pair's own label (or left to its default); or a `competing_noise` entry emitting that label	130[2]	The closed form sizes ℓ^* so that *all* of it sits inside k^*, and there is no search to correct a miss. Anything that can emit ℓ^* elsewhere silently delivers a different coefficient. Use `proportional_to_marginal`, or `scope: global`.
`balance` exists but is not defined	Only `balanced`/`unbalanced` is accepted	132[3]	Use lowercase `balanced` or `unbalanced`.

Table 3: Incompatibilities within or between several configurations reported as 1xx value errors as invalid configuration, but cannot be diagnosed by inspecting any single field.

[1] Not exit status 124 (`timeout` expired).
[2] Not exit status 130 (Ctrl-C or SIGINT).
[3] Not exit status 132 (illegal CPU instruction).

3 Ignored & Irrelevant Settings (expectations)

If the requirements are met with no incompatibilities in configuration (Section 4.1–4.3), all other mode×option mismatches are not considered as errors and are ignored. Some of them log a warning (see Section 4.4 for a complete reference); the others, such as an unread `skew_rule` or `skew_params`, are ignored silently.

Setting	Ignored / irrelevant when	Effect
`proportions`	`balance = balanced` (or absent), `matching_mode` ≠ `perfect`	Replaced by a uniform $1/M$ split; their length and sum are not checked (121/106). Warns 301.
`proportions`, `balance`, `skew_rule`, `skew_params`	`matching_mode = perfect`	Label counts are forced to each paired cluster's size. Neither read nor validated: a value another mode would refuse (106, 107, 121, 131, 132, 203) passes. Warns 302.
`skew_rule`, `skew_params`	`balance = unbalanced` *and* `proportions` given	Explicit `proportions` take precedence; `skew_rule` is the fallback only.
`skew_rule`, `skew_params`	`balance = balanced` (or absent)	Not read and not validated (107/131), and no warning.
per-row `recall_target`	`target_metric` is set	Overridden by the single solved global α. Warns 303.
`centroid_dependence`	`matching_mode = perfect`	Runs but has no visible effect: each cluster is 100% one label, so placement cannot discriminate.
`centroid_dependence`	`matching_mode = random`	Neither read nor validated: random places no point by its distance to a centroid, so data without feature vectors runs (no 125). Warns 311 unless it is switched off (`enabled: false` or absent).
`competing_noise`	`matching_mode = perfect`	No-op: zero unclaimed capacity to convert. Warns 305 per entry.
`split_rule`	a rule targets a single cluster	Short-circuited (the point mass goes to that one cluster directly). Still validated under `custom` (108), so a typo is reported even when no rule spans two clusters.
`split_rule`, `spillover_rule`	`matching_mode = random`	No per-cluster allocation step exists at all in random mode.
`steepness`	`profile` ≠ `exponential`	Unused (only the exponential weight uses it).
`concentrated_labels`	`spillover_rule` ≠ `concentrated`	Unused, and not validated in this case; default target is the single largest label. Under `concentrated` it *is* validated (128).

Table 4: Ignored configurations and their effect.

4 Error and Warnings Reference

4.1 Value Error (invalid configuration) 1xx

Code	Trigger	Reason	Mitigation
101	`matching_mode` is none of the four modes	Unrecognised mode string	Use `perfect`, `single`, `random`, or `custom`.
102	`perfect` with $M \neq K$	A bijection needs exactly one label per cluster	Set `num_classes` to the true cluster count, or switch to `custom`.
103	`single` with $M < 2$ or $K < 2$	Single needs a target label *and* at least one other, over ≥ 2 clusters	Raise `num_classes`; use data with ≥ 2 clusters.
104	`single_match.label` / a row `label` not an integer in $0..M-1$	The label id does not exist	Use a value in $0..$`num_classes`-1.
105	`single_match.cluster` / a row `cluster` not among the data's ids	Typo, or wrong id *type* (`byoc` ids are whatever your CSV column holds, and may be strings)	Use an id from the data; match the type (int vs string).
106	explicit `proportions` do not sum to 1 (tol. 10^{-6})	Not a probability distribution	Rescale so the entries sum to 1.
107	`skew_rule` not in {`geometric`,`dominant_minority`,`dirichlet`}	Unrecognised skew rule	Use one of the three, or give `proportions` directly.
108	`split_rule` not in {`proportional_to_size`,`equal`}	Unrecognised split rule	Use one of the two.
109	`spillover_rule` not in {`proportional_to_marginal`,`uniform`,`concentrated`}	Unrecognised spillover rule	Use one of the three.
110	`profile` not defined	Unrecognized placement profile.	Use one of the three.
111	`target_metric` under `perfect`, or under `custom` with `num_classes` $= 1$ or one cluster	No free `recall_target` to solve for; one class or one cluster leaves a single labeling, so every metric is fixed	Use `single`/`custom` with $M \geq 2$ and $K \geq 2$, or drop `target_metric`.
112	`target_metric.type` not `mcc`/`ari`	Only these two metrics are solvable	Set `type:` `mcc` or `ari`.
113	`target_metric.value` outside $[-1, 1]$	MCC/ARI live in $[-1, 1]$	Choose a value in range.
114	`target_metric` under `random`	No cluster–label structure to dial	Use `single`/`custom`.
115	`competing_noise` under `random`	No aligned label to compete with	Use `single`/`custom`.
116	`competing_noise.favors` not `core`/`boundary`/`random`	Unrecognised placement side	Use one of the three.
117	`competing_noise.share` outside $[0, 1]$	Share is a fraction of leftover points	Choose `share` in $[0, 1]$.
118	`competing_noise.label` outside $0..M-1$	The competing label id does not exist	Use a valid label id.
119	`competing_noise.cluster` not among the data's ids	Typo or wrong id type	Use an existing cluster id.
120	`target_metric`: every probed $\alpha \in [0, 1]$ is infeasible	No recall level fits this dataset. A warning above it names the levels that code blocked.	Apply the fix of the quoted code.
121	`proportions` length $\neq$ `num_classes`	The surplus entries became label ids that `num_classes` never declared, written into the dataset through `uniform`/`concentrated`.	Provide exactly `num_classes` proportions. Not raised under `balance: balanced` (or absent), where `proportions` are ignored (301), nor under `perfect` (302).

Code	Trigger	Reason	Mitigation
122	`target_metric.scope` not `pair`/`global`	Unrecognized scope string	Use `pair` or `global`.
123	`scope: pair` with `type: ari`	Only the single-pair MCC has a closed-form inverse	Use `type: mcc`, or `scope: global`.
124	`scope: pair` outside `single` mode	The pair is defined by `single_match`	Use `matching_mode: single`, or `scope: global`.
125	placement requested without feature vectors, or with one feature per point	Centroid distances need (N, d) coordinates, $d \geq 2$	Pass `coords`, or disable `centroid_dependence` and spatial `competing_noise`.
126	`num_classes` not an integer, or outside $[1, 64]$	Not an integer	Use an unquoted integer in $[1, 64]$.
127	the dataset's cluster count $K > 64$	Same backstop, applied to the data rather than the config	Regroup or subsample clusters to $K \leq 64$.
128	`concentrated_labels` is not a list of integers in $0..M-1$	A bare number is read as a *range* (scattering the remainder over that many labels).	Give a list of existing label IDs, e.g. `[0]`. Checked before the `target_metric` solver.
129	`centroid_dependence` mode `.favors` not `core`/`boundary`	Matched exactly and case-sensitively; any other value, including `Core` or a typo.	Use lowercase `core` or `boundary`. Only read when `enabled: true`.
130	`scope: pair` combined with a spillover or noise setting that can place the target label outside its cluster	`uniform` spillover, `concentrated` onto ℓ^*, and `competing_noise` break the closed-form assumption that every point of ℓ^* lies in k^*	Use `spillover_rule: proportional_to_marginal`, keep `competing_noise` off ℓ^*, or use `scope: global`.
131	`skew_params` out of range for the chosen `skew_rule`	Unchecked values could produced *negative* label counts that still summed to N	Use `geometric` `ratio` ≥ 0
132	`balance` not exactly `balanced`/`unbalanced`	Matched exactly and case-sensitively.	Use lowercase `balanced` or `unbalanced`, or omit the key.
133	A missing or infinite id, an id that is not an integer, or a `coords` row count different from N	numpy merged missing ids into one cluster that counted no points, and a Python list holding `1` and `"1"` into one cluster	Give every point one integer cluster id (`1`, not `1.0`, `"1"` or `true`), and one feature row.
141	a value of the wrong type (`null` included), or a block of the wrong shape	Values are not converted correctly.	Write the value unquoted and a cluster id as one integer.
142	A cluster named more than once for one label in two `assignment_matrix` or two `competing_noise` entries	One entry states how much of a label a cluster holds.	Name each cluster once per label: keep it in one row; for noise, choose one placement, or write one entry with the summed `share`.
144	A `competing_noise` entry whose label is a target of that cluster (its `single_match` or an `assignment_matrix`)	Competing noise places another label in a cluster; on the cluster's own target label the entry's `favors` moved the target's points too.	Use a different label, or remove the entry.

Table 5: Complete list of invalid configurations.

4.2 Infeasible Allocation Error 15x

Code	Trigger	Reason	Mitigation
150	A single rule: $\lfloor r \cdot m_\ell \rfloor >$ capacity of its clusters	The label's requested budget exceeds the target clusters' size	Lower `recall_target`, add clusters to the rule, or shrink the label.
151	Rules jointly claim more than a cluster's size, and no rounding of their counts to whole points fits	Overlapping rules oversubscribe one cluster	Lower the competing `recall_target`s or spread them across more clusters.
152[4]	`competing_noise` entries for one cluster claim more than its unclaimed points	Too much structured noise demanded on one cluster	Lower the `share` value(s).
153[5]	Several `assignment_matrix` rules name the same label and their `recall_target` values add up to more than 1	A rule's `recall_target` is a fraction of the label's *whole* budget, so repeating a label across rules adds those fractions.	Let `split_rule` divide the budget.
154	`target_metric` with `strict: true`, and the delivered value lies outside `value` ± `tolerance`	Strict refuses the closest value rather than writing it.	Choose a target below the ceiling or inside the range, give a `tolerance`, or omit `strict` to keep the closest label.

Table 6: Infeasible Allocation Error, *Under `target_metric`*, codes 150/151/152/153 are caught per probe and treated as "this α is infeasible". When the target is missed, each code is logged once as a warning naming the levels it blocked, before 306, or before 120 when no α works at all. 154 is only with `target_metric.strict`; the dataset is written without that label.

4.3 Key Error (a required key is missing) 2xx

Code	Missing key	Required when & fix
201	`num_classes`	Always. Add `num_classes: M`.
202	`matching_mode`	Always; written with no value, it is missing too. Add `matching_mode: <mode>`.
203	`skew_rule`	When `balance: unbalanced` and no `proportions`, in any mode but `perfect`. Any other `balance` value but `balanced` is refused first (132). Add a `skew_rule` or explicit `proportions`.
204	`single_match`	When `matching_mode: single`. Add `single_match:{cluster,label}`.
205	`cluster` / `label` inside `single_match`	Both sub-keys are required. Add the missing one.
206	`assignment_matrix`	When `matching_mode: custom`. Add at least one rule.
207	`label` / `clusters` inside a rule, `cluster` / `label` inside a `competing_noise` entry, or `clm_label` under `label_generation`	Every rule and entry needs both; a run that generates labels needs `clm_label`. Add the missing one.
208	`recall_target` inside a rule	In `custom` *without* `target_metric`. Add `recall_target`, or add `target_metric` to solve it globally.
209	`value` inside `target_metric`	When `target_metric` is present. Add `value: <target>`.

Table 7: Supplementary information for the context of KeyErrors.

[4]Not exit status 152 (CPU-time limit).
[5]Not exit status 153 (file-size limit).

4.4 Warnings (non-fatal) 3xx

Code	Trigger	Recommendation
301	`balance: balanced` (or absent) *and* `proportions` given, in any mode but `perfect` (302)	Expected. To use your `proportions`, set `balance: unbalanced`; otherwise remove them to silence the warning.
302	`perfect` with `proportions`, `balance`, `skew_rule` or `skew_params` set	Expected. Perfect fixes counts to cluster sizes and neither reads nor validates those keys; drop them to make intent clear.
303	`target_metric` present with per-row `recall_target`s	Expected. The global α overrides them; you may delete `recall_target` from the rows.
304	The delivered label counts differ from their targets, with `spillover_rule uniform`/`concentrated`	Expected: the label marginal is no longer binding, so achieved counts are not held to `proportions`. Checked against the delivered counts after placement, so it does not fire when they match. The message names the rule and gives both count vectors. For exact counts use `spillover_rule: proportional_to_marginal`; `competing_noise` then places at most what the matching leaves of a label's count (305).
305	A `competing_noise` entry places fewer points than its `share` asks, or none, in the delivered labeling (a `target_metric` search probe does not report it)	The cluster is fully claimed, the `share` rounds to 0, or, under `proportional_to_marginal`, the label has less of its count left after the matching than its entries ask, and they place what is left. One code, three causes; the message names which and how many points the entry placed. Lower a rule's `recall_target`, or choose another cluster or label.
306	`target_metric` did not converge within tolerance	The target exceeds the structural ceiling (e.g. $M \ll K$). When nothing can move the label counts, the message names their size ceiling (for ARI, a bound that is not always reached), which no labeling with those counts exceeds. An MCC target below 0 is named as unreachable: the matched R_K is never negative. The closest feasible value is returned (with `strict: true`, refused by 154); lower the target, raise `max_iter`/`tolerance`, or change `proportions`/clusters. Raising `max_iter` helps only when every iteration is used.
307	`scope: pair` target outside the pair's reachable range	The request is clamped to the nearest reachable value; the message reports the range, choose a target within it.
308	`scope: pair` resizes the target label	Expected: the label is sized as a subset of its cluster so the pair MCC meets the target exactly; an explicit proportion for that label is overridden. It does not fire when that size equals the label's planned count.
309	the delivered `target_metric` is outside tolerance and differs from what the search scored at the same α	An explicit `target_metric.probe_seed` differs from the run seed, so the search scored a labeling other than the one written. Omit `probe_seed`; it defaults to the run seed. With `strict: true`, 154 then refuses the label.
310	`scope: pair`: the delivered pair MCC is outside the request	The counterpart of 306 for the closed form. The message names the cause: the label holds a whole number of points, so the reachable values step past the target, or the target lies outside the reachable range (307). The combinations that would move the label out of its cluster are rejected up front (130). With `strict: true`, 154 refuses the label instead.
311	`random` with `centroid_dependence` switched on	Expected. Random draws labels with no regard to the clusters, so no point is placed by its distance to a centroid; the block is neither read nor validated. Remove it or set `enabled: false`; to place labels by centroid, use `single`/`custom`.

Table 8: Warning codes, which indicate scenarios of successfully executed configuration, but with a different outcome than the user would expect.

5 Uncoded Rejections

Not every rejection is a CLM diagnostic. The banded codes above describe the *cluster-label matching model*: what a configuration asks the engine to do, and whether that is coherent. The conditions in this section describe something prior, whether the *data* that was handed to the engine is a usable clustering at all.[6]

5.1 BYOC input requirements

BYOC is an *import* path, not a generator. The premise is that the user has already clustered a subset of their features with their own algorithm (k-means, fuzzy c-means, or any other) and is bringing the result: the feature subset in which the clustering was computed, plus one column holding each point's cluster. The requirements in the following are derived from that premise rather than from the CLM engine.

Every requirement is checked in a single pass, so a file with several problems reports all of them at once rather than one per attempt. Any failure rejects that dataset; under a multi-dataset configuration, the remaining datasets are unaffected and independently checked.

Requirement	Rejected when	Why
Empty file	No data rows, or no columns	A header alone is not a clustering.
Distinct column names	Any name appears twice	`pandas` renames the second to `name.1`, so the column meant is ambiguous, and which of the two originals it holds depends on their order in the file.
No reserved names	A column is named `Cohort_Class`, `GroundTruth_*`, `Cluster_n` or `Label_n`	The first two are consumed as ground truth and would silently disappear as features; the last two are written into the output, producing a duplicate column in the result.
Declared cluster column	`cluster_column` missing, not a single name, or absent from the file	The ground-truth partition must be named explicitly; it is never guessed.
Complete cluster column	Any missing value	Every point belongs to a cluster. A blank would become a cluster of its own and change K.
At least two clusters	The cluster column holds one distinct value	A single partition has no structure for a label to agree or disagree with.
Minimum cluster size	Any cluster holds fewer than 3 points	A cluster of one or two points is a stray rather than a cluster.
Integer cluster ids	Any id is not an integer (`1.0`, `C1`, `true`)	Clusters are unsupervised: an id only tells which points belong together. A name reads as a human label, and human labels are artificially correlated and may be wrong. The column's name is free.
Numeric features	Any non-cluster column is non-numeric	Every column other than the cluster column is treated as a feature, which define the geometry the clustering was computed from.
Complete features	Any missing value in a feature column	The clustering cannot have been computed from missing values, and centroid distances would evaluate to `NaN` without complaint.

Table 9: BYOC import requirements. These reject a dataset before any label generation; they are not CLM codes, because they concern the data rather than the cluster-label matching program.

***Not* restricted.** The cluster column may have any name. The names of the feature columns are carried verbatim to the output CSV and plot axes. Repeated identical rows are kept, since they are additional points rather than a data error, and removing them would change the cluster sizes on which the matching arithmetic is built.

[6] This section is expected to grow as more input paths gain checks, and it currently covers `data_source: byoc`.

CLMSynth · run pipeline

clmsynth <config> → YYYYMMDD_HHMMSS_Source/{csv, png, info} · one process per config

CLMSynth v0.7.5rc1

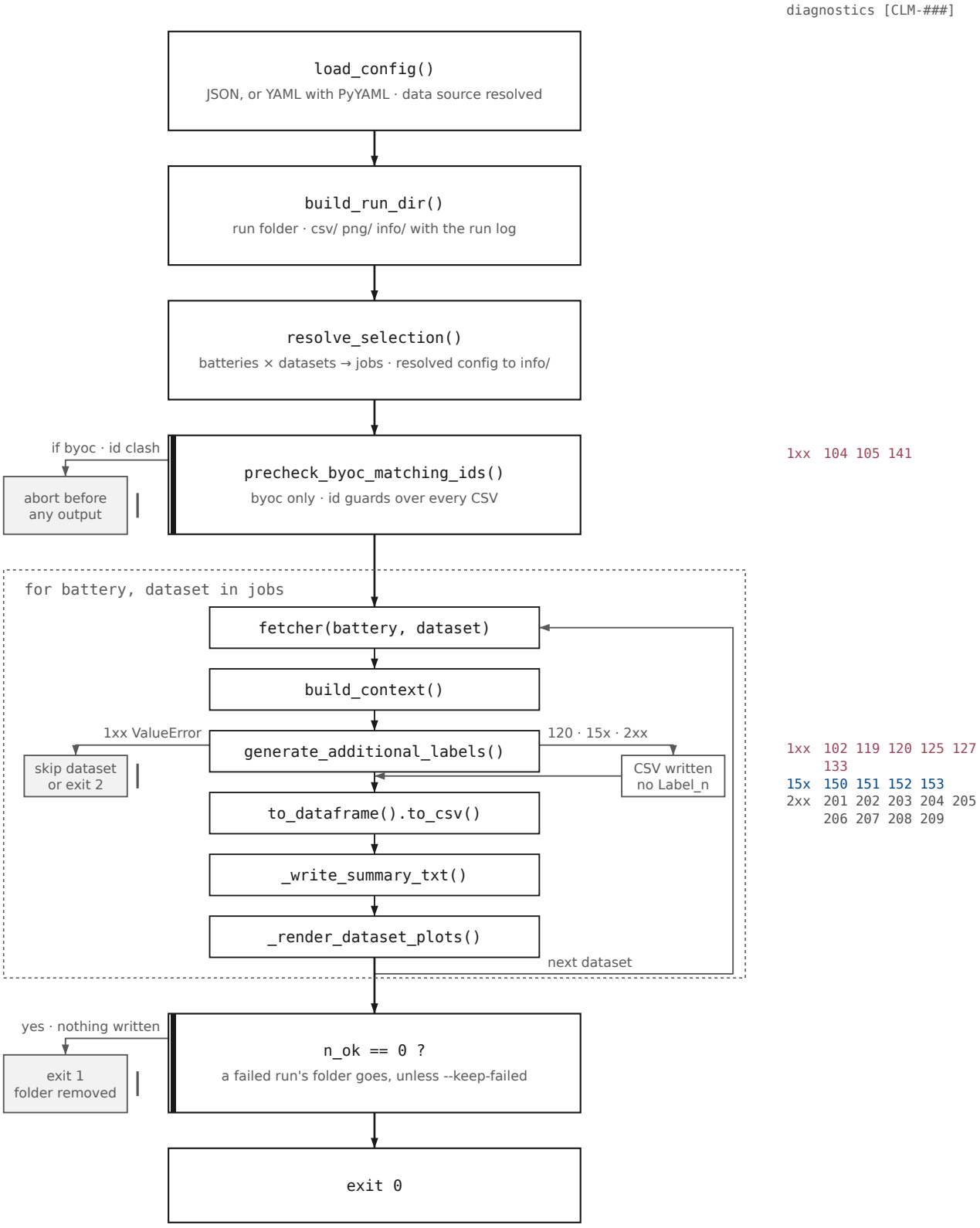


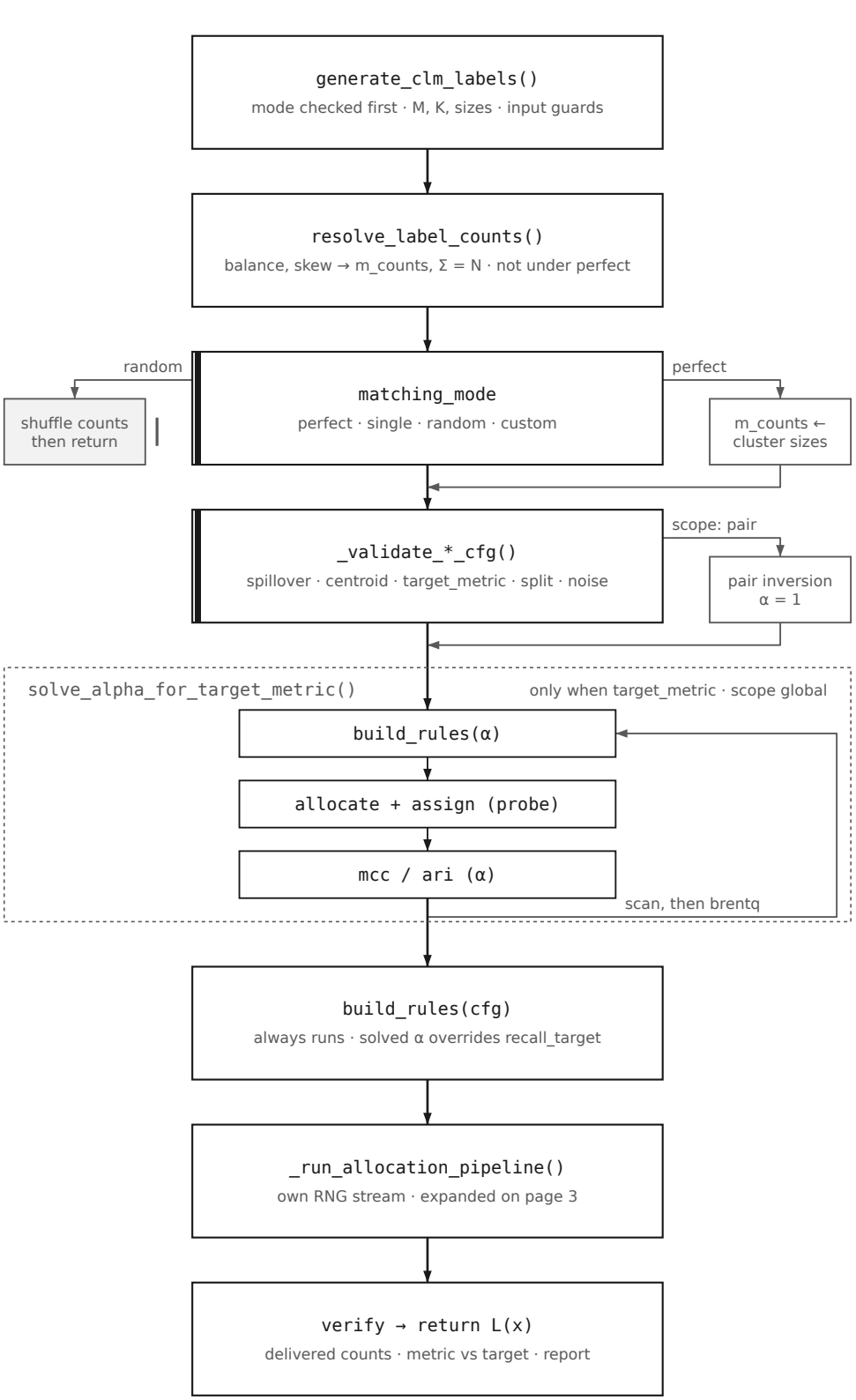
Figure A.1 · outer pipeline (1 of 3)

1xx invalid config · 15x infeasible allocation · 2xx missing key · 3xx warning

CLMSynth · clm_label_engine

c(x), X, cfg, seed → L(x) ∈ {0 … M−1}, deterministic under seed

CLMSynth v0.7.5rc1



diagnostics [CLM-###]

generate_clm_labels()	1xx 101 102 125 126 127 133 141 2xx 201 202
resolve_label_counts()	1xx 106 107 121 131 132 2xx 203 3xx 301
matching_mode	1xx 114 115 3xx 311
_validate_*_cfg()	1xx 108 110 111 112 113 116 117 118 122 123 124 128 129 130 144 2xx 209 3xx 303 307 308
solve_alpha_for_target_metric()	1xx 120 3xx 306
build_rules(cfg)	1xx 103 104 105 142 2xx 204 205 206 207 208 3xx 302
verify → return L(x)	15x 154 3xx 304 309 310

continues · stops here · branches

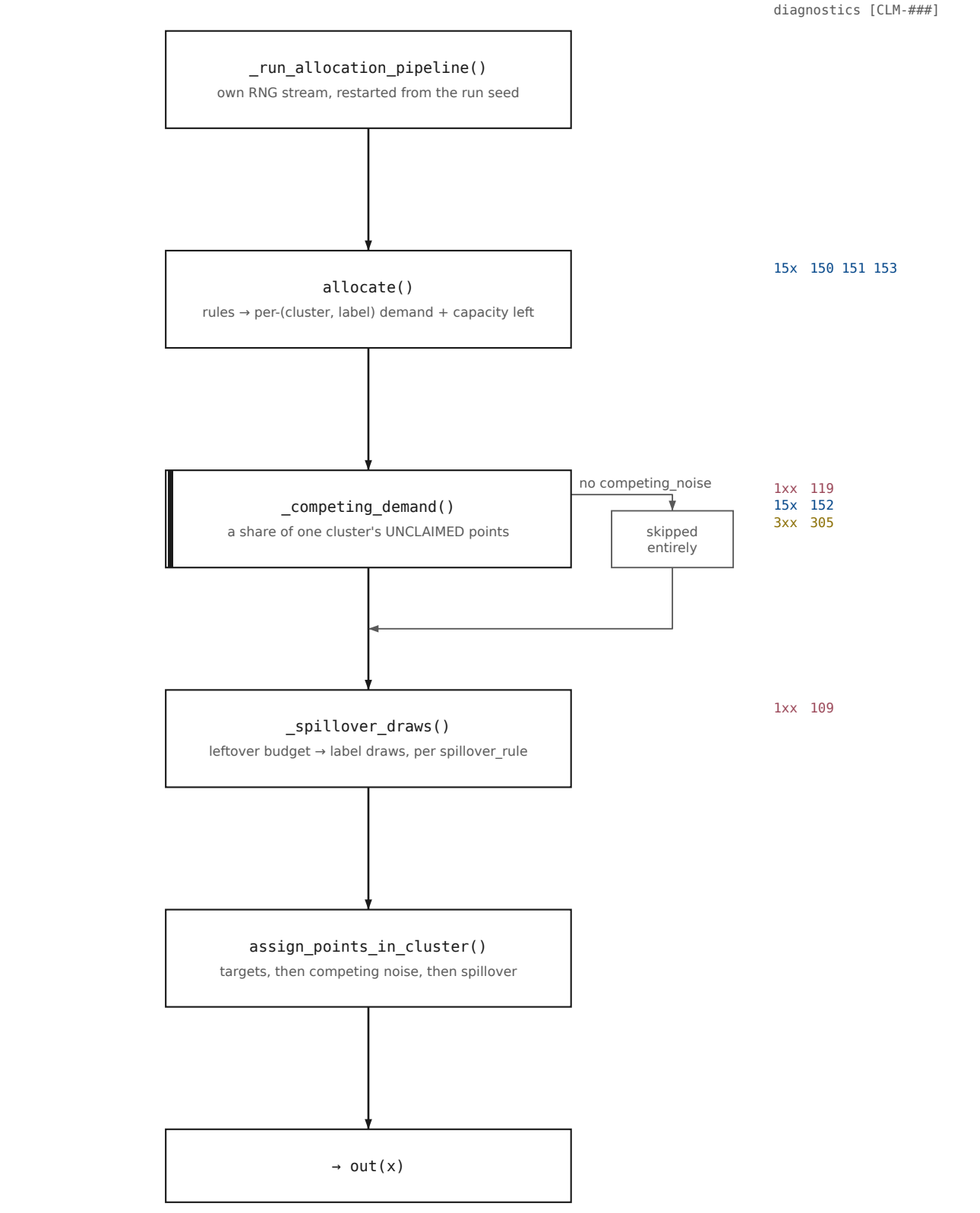
run or dataset stops · returns to the spine · one step, expanded

Figure A.2 · engine internals (2 of 3)

1xx invalid config · 15x infeasible allocation · 2xx missing key · 3xx warning

CLMSynth · _run_allocation_pipeline

rules, m_counts, cluster_sizes → out(x), one label per row



continues · stops here · branches

run or dataset stops · returns to the spine · one step, expanded

Figure A.3 · allocation and placement (3 of 3)

1xx invalid config · 15x infeasible allocation · 2xx missing key · 3xx warning